

TrainTracks — Complete Project Documentation

Version: 1.0 · **Date:** March 29, 2026
Framework: Next.js 14.1 · **Language:** TypeScript 5 · **Styling:** TailwindCSS 3.4 · **State Management:** Zustand 4.5

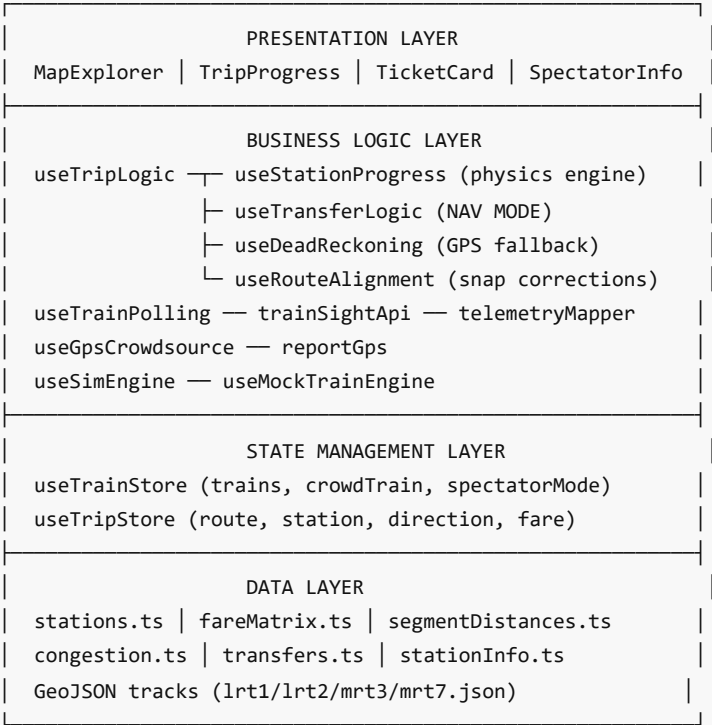
1. Architecture Overview

TrainTracks is a real-time Metro Manila rail navigation PWA that provides live train tracking, GPS-powered trip guidance, and crowdsourced telemetry for LRT-1, LRT-2, MRT-3, and MRT-7.

Core Design Principles

- Route-First Architecture:** All navigation uses a pre-computed `Station[]` route. Station advancement is atomic via `advanceToStation()`.
- Telemetry Unification:** Both live API data and local simulation produce identical 4-stage status strings: `CURRENT STATION` → `NOW LEAVING` → `IN TRANSIT TO` → `NOW APPROACHING`.
- Visual Progress Decoupling:** Progress bar refs lag behind the store to prevent premature 0% resets during station capture.
- Crowdsourcing Merge:** The train store automatically merges persistent `crowdTrain` state into every API poll cycle, suppressing duplicate echoes.

System Layers



2. Root Configuration

package.json

Key Dependencies:

Package	Version	Purpose
next	14.1.0	React framework (App Router)
react / react-dom	18.2.0	UI library
zustand	^4.5.0	Lightweight state management
leaflet / react-leaflet	^1.9.4 / ^4.2.1	Map rendering
@supabase/supabase-js	^2.96.0	Auth & database
framer-motion	^12.33.0	Animations
lucide-react	^0.300.0	Icon library
@ducanh2912/next-pwa	^10.2.9	Progressive Web App
@turf/turf	^6.5.0	Geospatial (mostly superseded by native Haversine)
socket.io-client	^4.8.3	WebSocket (reserved)

Other Config Files

File	Purpose
tsconfig.json	TypeScript config with @/ path alias → ./src/*
next.config.mjs	PWA configuration via @ducanh2912/next-pwa
tailwind.config.ts	Custom colors: bg-lrt1, bg-lrt2, bg-mrt3. TailwindCSS Animate plugin
middleware.ts	Supabase auth middleware — refreshes session tokens on every request
.env.local	Supabase URL/anon key, TrainSight API token/client ID
eslint.config.mjs	ESLint config (Next.js preset)
postcss.config.js	PostCSS + TailwindCSS + Autoprefixer

3. Type Definitions (src/types/)

index.ts (355 bytes)

Core types used throughout the application:

```
type LineId = 'LRT1' | 'LRT2' | 'MRT3' | 'MRT7';

interface Station {
  id: string;           // e.g., "L1-20", "M3-01"
  name: string;         // e.g., "Roosevelt (FPJ)"
  lineId: LineId;
  order: number;       // Sequential position on line
}
```

```
latitude: number;
longitude: number;
transfers?: LineId[]; // Lines accessible via walking transfer
isUnderground?: boolean;
}

interface Coordinates {
latitude: number;
longitude: number;
}
```

train.ts (1,857 bytes)

TrainSight API type definitions:

```
interface TrainPosition {
train_id: string; // "9aa3053c-938" (live) | "SCHED-xxx" | "CROWD-xxx" | "MOCK-xxx"
lat: number;
lng: number;
line: string; // "LRT-1", "LRT-2", "MRT-3"
direction: string; // "Northbound", "Southbound", "Eastbound", "Westbound"
speed: string; // "40 km/h"
status: string; // "IN TRANSIT TO: [L1-09] Bambang"
}

interface GpsReportPayload {
lat: number;
lng: number;
speed: number;
client_id: string;
vehicle_type: number; // 1 = train
is_authoritative: boolean; // false = crowdsourced
}
```

Line Name Mappings:

- TRANSIGHT_LINE_MAP : { 'LRT-1': 'LRT1', 'LRT-2': 'LRT2', ... } (API → Internal)
- INTERNAL_LINE_MAP : { 'LRT1': 'LRT-1', ... } (Internal → API)

4. Data Layer (src/data/)

stations.ts (10,048 bytes)

Master station database — **65 stations** across 4 lines:

Line	Stations	Range	Notable
LRT-1 (Green)	25	L1-20 Roosevelt → L1-25 Dr. Santos	Includes Cavite Extension (5 stations)

LRT-2 (Purple)	13	L2-01 Recto → L2-13 Antipolo	Katipunan is underground
MRT-3 (Yellow)	13	M3-01 North Avenue → M3-13 Taft Avenue	Buendia & Ayala underground
MRT-7 (Red)	14	M7-01 Common Station → M7-14 San Jose del Monte	Under construction (DevOpts)

Transfer Stations: D. Jose↔Recto (LRT1↔LRT2), EDSA↔Taft Avenue (LRT1↔MRT3), Cubao↔Cubao (LRT2↔MRT3), Common Station (MRT7↔LRT1/MRT3).

LINES constant: Physical train specifications per line:

- Operating/design speed, average commercial speed
- Train car count, train/car length in meters
- MRT-3 rush hour override (4 cars instead of 3)

getFilteredStations(mode) : Filters based on MRT-7 DevOpts:

- 'OFF' : Hide all MRT-7 stations
- 'WITH_NA' : Show all (North Avenue + MRT-7 coexist)
- 'WITHOUT_NA' : MRT-7 present but North Avenue removed

fareMatrix.ts (73,955 bytes)

Complete fare lookup: `FARE_MATRIX[originId][destId] = { SJT: number, BEEP: number }` . Covers all station pairs. Sourced from official LRTA/LRMC/DOTr CSV data.

segmentDistances.ts (7,003 bytes)

Real-world track distances (km) between adjacent stations. Used for progress calculations instead of Haversine (which underestimates on curved tracks). Key export: `getSegmentDistanceKm(stationA, stationB): number | null` .

congestion.ts (24,678 bytes)

Rush hour congestion model based on DOTr ridership data:

- `getRushHourData(stationId, dayOfWeek, hour)` → congestion tier
- `getCongestionLevel(stationId)` → real-time from current time
- Tiers: EXTREME / HIGH / MODERATE / LOW

transfers.ts (4,078 bytes)

Walking transfer details:

- `getTransferDetails(fromLine, toLine, stationName)` → `{ distanceMeters, walkTimeMinutes, direction, instruction, routeDescription, targetCoordinates }`
- D.Jose↔Recto: 340m, ~5min; EDSA↔Taft: 150m, ~2min; Cubao↔Cubao: 200m, ~3min

stationInfo.ts (55,022 bytes)

Rich metadata for Line Explorer: address, landmarks, accessibility, connecting transport, operating hours, historical facts.

geofence.ts (1,139 bytes)

Metro Manila polygon boundary for RegionGuard.

GeoJSON Track Files

- 1rt1.json (11,112 bytes), 1rt2.json (4,369 bytes), mrt3.json (3,857 bytes), mrt7.json (5,502 bytes)
- Track polylines for map rendering via Leaflet GeoJSON layers.

5. Utilities (src/utils/)

geo.ts (4,409 bytes) — Geospatial Math

Function	Signature	Purpose
getDistanceKm	(from, to) → number	Haversine great-circle distance
getProgress	(start, end, current) → number	Linear progress % via Law of Cosines projection. Snaps to 100% at 30m
getStationProgress	(start, end, current) → number	Real track distance version — uses segmentDistances.ts, falls back to Haversine
moveTowards	(current, target, distKm) → Coordinates	Interpolate position along bearing by distance
getBearing	(start, end) → number	Compass bearing in degrees (0-360)

stationUtils.ts (5,715 bytes) — Display Helpers

Function	Purpose
getStationBadge(lineId, order)	Badge string: GL01 (LRT1), PL05 (LRT2), YL13 (MRT3), RL02 (MRT7)
getLineColor(lineId)	TailwindCSS bg class: bg-1rt1, bg-1rt2, bg-mrt3, bg-red-700
getThemeColors(lineId)	Complete glassmorphism theme object: glass, border, shadow, accent, text colors
getDoorSide(stationName, lineId)	'LEFT' / 'RIGHT' / 'EITHER' — per-station door opening side

fareNew.ts (4,334 bytes) — Fare Engine

Function	Purpose
getPrecisionFare(origin, current, ticketType)	Exact fare from matrix. Cross-line via TRANSFER_MAP recursive sum
getFareBreakdown(origin, current, ticketType)	Split by line: { 1rt1, 1rt2, mrt3, total }

Ticket types: SJT (Single Journey), SVC (Beep stored value), CONCESSION (50% off SJT), DEBIT , CREDIT .

simRoute.ts (4,001 bytes) — Route Planner

Function	Purpose
<code>getRoute(origin, dest, options?)</code>	Returns <code>Station[]</code> path. Handles same-line, 1-transfer, and 2-transfer routes
<code>getLineSegment(start, end)</code>	Ordered station slice on one line

TRANSFER_MAP (hardcoded pairs):

- LRT1↔LRT2: D. Jose (L1-11) ↔ Recto (L2-01)
- LRT1↔MRT3: EDSA (L1-02) ↔ Taft (M3-13)
- LRT2↔MRT3: Cubao (L2-08) ↔ Cubao (M3-04)

telemetryMapper.ts (5,566 bytes) — API Status Interceptor

Transforms raw TrainSight API status strings into the unified 4-stage format:

Raw API Status	Mapped Status
"Scheduled: Heading to NAV-MRT3"	"IN TRANSIT TO: [M3-01] North Avenue"
"Dwelling at FPJ"	"CURRENT STATION: [L1-20] Roosevelt (FPJ)"

Status Logic (progress-based, not distance):

- Progress < 12% → NOW LEAVING: [previous station]
- Progress 12–85% → IN TRANSIT TO: [target]
- Progress > 85% → NOW APPROACHING: [target]
- Speed = 0 at station → CURRENT STATION: [target]

6. Services (src/services/)

trainSightApi.ts (2,946 bytes)

Centralized API client for `https://core.trainsight.app` :

Function	Endpoint	Returns	Purpose
<code>fetchTrains()</code>	GET /api/trainsight/trains (proxied)	<code>TrainPosition[]</code>	All active train positions, auto-enhanced via <code>telemetryMapper</code>
<code>fetchFleetSize()</code>	GET /api/fleetSize	<code>Record<string, number></code>	Fleet count per line
<code>reportGps(payload)</code>	POST /api/trainsight/gps (proxied)	<code>boolean</code>	Crowdsourced GPS to API. Fire-and-forget

7. State Management (src/store/)

useTrainStore.ts (4,637 bytes)

Field	Type	Purpose
<code>trains</code>	<code>TrainPosition[]</code>	All active trains (live + mock + crowd)

spectatorMode	boolean	Companion vs Spectator toggle
selectedTrainId	string null	Train being inspected
followedTrainId	string null	Camera-locked train
mockTrainsMode	boolean	Ghost Train simulation active
isBroadcasting	boolean	GPS crowdsourcing active
crowdTrain	TrainPosition null	Persistent crowdsourced marker
timeFactor	number	Simulation speed multiplier

Critical Action — setTrains(incoming) :

1. Reads persistent crowdTrain from state
2. Filters out any SCHED- or LIVE train on same line+direction within ~1.1km (0.01°)
3. Injects crowdTrain into final array
4. Ensures CROWD- marker survives every 1s polling cycle

useTripStore.ts (13,863 bytes)

Group	Fields
Selection	selectedLine, origin, destination, ticketType
Route	computedRoute: Station[], routeIndex: number
Dynamic	currentStation, nextStation, status (IDLE/WAITING/TRANSIT/ARRIVED), direction, runningFare
Navigation	walkingDistance, simulatedLocation/Heading/Speed, isGpsOverride
Preferences	isMuted, notificationPreference, isDarkMode, showRushHour
DevOpts	isDevMode, line7Mode (OFF/WITH_NA/WITHOUT_NA)

Key Actions:

- startTrip() : Computes route, initializes stations, sets status: 'WAITING'
- advanceToStation(id) : THE atomic mutation — sets station, increments routeIndex, updates direction + fare
- endTrip() : Resets all state, saves history
- recomputeRoute() : Rebuilds route from current origin+destination

8. Hooks (src/hooks/)

useTripLogic.ts (7,038 bytes) — ★ THE ORCHESTRATOR

Fuses 4 sub-hooks into unified output:

```
useTripLogic
├─ useStationProgress → statusText, legProgress, displayStation
├─ useTransferLogic   → NAV MODE overlay (status, walking progress)
```

```
└─ useDeadReckoning → GPS fallback flag
└─ useRouteAlignment → route snap corrections
```

Also handles: GPS heading → direction sync (speed-gated ≥ 15 km/h, terminus-guarded), route-distance total progress, stops remaining calculation.

useSmartLocation.ts (6,753 bytes) — GPS Abstraction

- GPS Override support (returns simulated data for simulation mode)
- Heading inference: compass → NORTH / SOUTH (or EAST / WEST for LRT-2)
- **Watchdog Timer:** Auto-restarts GPS if no updates for 15s (iOS Safari fix)

useTrainPolling.ts (3,265 bytes)

Polls `fetchTrains()` every 1 second. Auto-paused when not in view.

useGpsCrowdsource.ts (7,428 bytes) — GPS Broadcasting

Two independent concerns:

1. **CROWD Marker:** Persists `CROWD-` train via `setCrowdTrain()` for entire trip duration. Builds telemetry status strings matching API format.
2. **API Reporting:** Sends GPS every 3s during motion states only (speed ≥ 5 km/h).

useSimEngine.ts (19,032 bytes) — Ghost Train Engine

Generates virtual train on user's route:

- Synthetic GPS via `moveTowards()`
- 4-stage status state machine with dwell times
- Zero-latency arrival detection (check after movement in same tick)
- `timeFactor` speed multiplier support

useMockTrainEngine.ts (13,351 bytes)

SCHED- prediction engine — generates fake trains across all lines for spectator map when no live data.

useWrongDirection.ts (14,412 bytes)

- Compares heading vs expected route direction
- Transfer suppression, speed gate (≥ 15 km/h), terminus guard
- LRT-2 compass fix (East/West)
- Multi-confirmation (3+ readings required)

useStallDetector.ts (6,719 bytes)

Alerts when speed = 0 during transit for extended duration. Distinguishes station dwelling from mid-track stalls.

useCongestionAlert.ts (2,954 bytes)

Rush hour crowding alert based on congestion data.

Other Hooks

Hook	Size	Purpose
<code>useAuth.ts</code>	7,388	Supabase auth (sign-in/up/out, session)

useBackgroundAudio.ts	2,209	Silent audio to keep app alive on iOS
useDeviceOrientation.ts	3,827	Compass heading (handles iOS permission)
useGatekeeper.ts	3,475	Station geofence entry (150m radius)
useOperatingHours.ts	1,349	Rail operating hours (4:30 AM – 10:30 PM PHT)
useTripHistory.ts	5,221	Trip persistence to Supabase
useTripNotifications.ts	5,488	TTS announcements (station, transfer, arrival)
useWakeLock.ts	1,767	Screen Wake Lock API

9. Trip Sub-Hooks (src/hooks/trip/)

useStationProgress.ts (18,945 bytes) — ★ THE PHYSICS ENGINE

The most complex hook (405 lines). Implements the station detection state machine:

Status States (hysteresis-locked, forward-only):

CURRENT STATION (0) → NOW LEAVING (1) → IN TRANSIT (2) → NOW APPROACHING (3) → [captures next]

Key Mechanisms:

- **Segment-Aware Zones:** Radii scale with actual segment distance (adaptive)
- **Visual Decoupling:** progressOriginRef / progressTargetRef lag behind store
- **High-Water Mark:** Progress only increases (prevents GPS drift regression)
- **Soft Arrival:** Smoothly reaches 100% at boundaries

useTransferLogic.ts (9,361 bytes) — NAV MODE

Rules:

1. Entry: CURRENT STATION at a transfer station
2. Walking distance via Haversine (meters)
3. Progress via getStationProgress() with HWM lock
4. Exit: ≤30m from partner AND GPS accuracy < 150m
5. On exit: advanceToStation() atomically switches line, status → WAITING

useDeadReckoning.ts (9,659 bytes) — GPS Fallback

Extrapolates position from last speed + heading when GPS drops. Auto-recovers when signal returns.

useRouteAlignment.ts (3,901 bytes)

Snaps routeIndex to closest station when GPS drifts off computed route.

10. Components (src/components/)

Core Application Components

Component	Size	Purpose
-----------	------	---------

MainApp.tsx	37,458	Root orchestrator — map, trip logic, simulation, crowdsourcing, UI layout
MapExplorer.tsx	39,461	Leaflet map — GeoJSON tracks, station markers, dwelling blooms, rush hour overlays, camera control
LiveTrainLayer.tsx	13,905	Train marker management — smooth interpolation, tooltips, source labels, click-to-select
TripProgress.tsx	28,733	Trip HUD — stats cycling (distance/door/direction), speed counter, transfer compass, fare breakdown, progress bar
TicketCard.tsx	37,158	Trip setup — station search, line/ticket selection, fare preview, favorites
CommandCenter.tsx	28,857	Simulation controls — Ghost Train speed, route viz, dev toggles, GPS override
SpectatorInfoCard.tsx	26,564	Train/station info panel — source badges, formatted status, follow button, upcoming trains
UpcomingTrainsCard.tsx	13,960	Approaching/departing trains — filters by station, source badges, hides during transit

Navigation & Alert Components

Component	Size	Purpose
GeofenceScanner.tsx	12,772	Auto-detect nearby stations, suggest origin
LineExplorer.tsx	19,295	Full station directory with rich info
FareSelector.tsx	18,157	Ticket type selection with animated cards
WrongDirectionAlert.tsx	7,798	Full-screen red alert, dismiss + safe mode
CongestionAlert.tsx	6,594	Rush hour tier-based crowding alert
StallAlert.tsx	7,583	Train stall notification
StationInfoModal.tsx	11,001	Detailed station info (accessibility, landmarks)
StationTimeline.tsx	2,657	Vertical route timeline
NearbyStationsCard.tsx	4,393	GPS-sorted nearby stations
RecentTripsCard.tsx	6,729	Trip history display
TrackingCard.tsx	3,111	GPS status card
ReconnectionBanner.tsx	3,101	GPS reconnection indicator
GPSFallbackHandler.tsx	3,029	Dead reckoning visual indicator

Auth & Profile Components

Component	Size	Purpose
-----------	------	---------

AuthModal.tsx	25,880	Sign-in/sign-up form
AuthGate.tsx	3,860	Route protection guard
ProfileDrawer.tsx	12,976	Settings side drawer
ProfileSetupModal.tsx	24,173	First-time profile setup
LoginSuccessModal.tsx	4,929	Post-login animation
LogoutModal.tsx	10,683	Logout confirmation
ChangeEmailModal.tsx	14,407	Email change with verification
ChangePasswordModal.tsx	20,529	Password change with validation
NotificationSettingsModal.tsx	9,161	TTS/notification preferences
TripHistoryModal.tsx	24,066	Full trip history browser

Dev & Security Components

Component	Size	Purpose
AdminDashboard.tsx	17,818	Admin panel (fleet, stats)
DevControls.tsx	2,340	DevOpts toggle button
DevDashboard.tsx	5,038	Debug GPS display
SecurityGuard.tsx	15,285	DevTools detection + DOM scrubbing
RegionGuard.tsx	3,582	Metro Manila geofence
ErrorBoundary.tsx	2,736	React error boundary
PanicLogger.tsx	2,323	Global error handler
InfoCard.tsx	1,552	Generic card wrapper
NoSSR.tsx	281	Client-only render wrapper

11. UI Primitives (src/components/ui/)

Component	Size	Purpose
LoadingScreen.tsx	8,347	Full-screen loading with train icon animation
Marquee.tsx	1,692	Auto-scrolling text for long station names
badge.tsx	1,050	CVA badge variants
button.tsx	2,117	CVA button variants
card.tsx	1,443	Card container (header/content/footer)

dialog.tsx	4,282	Radix UI modal dialog
------------	-------	-----------------------

12. Screen Components (src/components/screens/)

Component	Size	Purpose
WelcomeScreen.tsx	9,734	First-time onboarding flow
ClosedScreen.tsx	3,582	Outside operating hours display
SafetyRules.tsx	5,860	Rail safety information
TransitRules.tsx	6,007	Transit etiquette & rules

13. App Routes (src/app/)

Layout & Pages

File	Purpose
layout.tsx	Root layout: Cabin font, PanicLogger, SecurityGuard, RegionGuard, ErrorBoundary
page.tsx	Landing page → NoSSR-wrapped MainApp
globals.css	Tailwind directives + custom scrollbar + map CSS
error.tsx	Error page
global-error.tsx	Root error boundary
not-found.tsx	404 page

Route Directories

Route	Purpose
/login	Auth page
/admin	Admin dashboard
/explorer	Line explorer
/profile	User profile
/profile/setup	First-time setup
/auth/callback	Supabase OAuth callback

14. API Routes (src/app/api/)

/api/trainsight/trains — Train Positions Proxy

Proxies to `POST https://core.trainsight.app/api/data/trains` . Injects API key server-side to avoid CORS and token exposure.

/api/trainsight/gps — GPS Crowdsourcing Proxy

Proxies to `POST https://core.trainsight.app/api/v1/gps/ingest` . Injects auth token. Returns 200 for rate-limited (429) responses.

/api/tts — Text-to-Speech

Server-side TTS generation for station announcements.

15. Root Data & Utility Files

File	Purpose
lrt1-sjt-fares.csv / lrt1-beep-svc-fares.csv	Source fare data for LRT-1
lrt2-sjt-fares.csv / lrt2-beep-svc-fares.csv	Source fare data for LRT-2
mrt3-sjt-fares.csv / mrt3-beep-svc-fares.csv	Source fare data for MRT-3
lrt1-geopoint.xhr / lrt2-geopoint.xhr / mrt3-geopoint.xhr	Raw GeoJSON track data
mrt7_coords.json	MRT-7 station coordinates
api_docs.txt / api_pages/	TrainSight API documentation
Quick Commuter Crowd Guide.txt	Rush hour crowding reference
wiki_results.txt	Scraped station info
Beep_Card_white_stacked.webp	Beep card image asset
fetch_mrt7_coords.mjs / fetch_wiki.mjs / patch_geojson.mjs	One-time data collection scripts
test-crowd.ts / test-math.ts / test-route.ts	Validation test scripts

16. File Statistics

Category	Files	Total Size
Types	2	~2.2 KB
Data	11	~195 KB
Utilities	5	~24 KB
Services	1	~3 KB
Stores	2	~18.5 KB
Hooks (top-level)	17	~110 KB
Trip Sub-Hooks	4	~42 KB

Components	40	~545 KB
UI Primitives	6	~19 KB
Screens	4	~25 KB
App Routes/Pages	7	~10 KB
TOTAL	~99 files	994 KB (1 MB)

End of Documentation